

Guía del caso aplicado

Sistema de Gestión y Consulta de Biblioteca Académica

Integración de desarrollo web, complejidad algorítmica, recursión, ordenamiento y búsqueda.

Entrega	Modalidad	Tecnologías	Soporte
En dos semanas	Grupos de 3	React + FastAPI	GitHub + demo

Propósito del proyecto

Desarrollar una aplicación web que permita registrar, consultar, buscar, ordenar y explorar libros académicos mediante una interfaz en React y una API en FastAPI. El proyecto debe evidenciar el uso de algoritmos de ordenamiento, búsqueda y al menos una solución recursiva con su respectivo análisis de complejidad.

1. Descripción del caso

La biblioteca de una institución universitaria necesita un prototipo web para organizar su catálogo académico y facilitar la consulta de libros por parte de estudiantes y docentes.

El sistema debe permitir gestionar libros, mostrarlos en una interfaz gráfica, aplicar búsquedas y ordenamientos, y representar una jerarquía de categorías como árbol. Un ejemplo de esa jerarquía es: Ingeniería > Programación > Desarrollo Web.

La meta no es construir una plataforma comercial completa, sino una solución funcional y bien argumentada desde el punto de vista de estructuras de datos y algoritmos.

Producto esperado

- Una aplicación web funcional con frontend en React y backend en FastAPI.
- Un catálogo persistente de libros académicos.
- Un módulo de búsqueda y un módulo de ordenamiento implementados de forma explícita.
- Una funcionalidad recursiva asociada a la jerarquía de categorías.
- Una explicación clara de la complejidad temporal de las soluciones implementadas.

2. Requerimientos funcionales mínimos

El proyecto deberá incluir como mínimo los siguientes módulos y comportamientos:

Estructuras de Datos y Algoritmos 1

Módulo	Requerimiento	Evidencia esperada
Backend	API con FastAPI para crear, listar, consultar, actualizar y eliminar libros.	Endpoints funcionales y probados.
Frontend	Interfaz en React para visualizar el catálogo y consumir la API.	Pantallas o componentes funcionales.
Búsqueda	Módulo para encontrar libros por título, autor, ISBN o categoría.	Barra de búsqueda y resultados correctos.
Ordenamiento	Ordenar por título, año, autor o disponibilidad.	Selector de criterio y salida ordenada.
Recursión	Explorar o contabilizar categorías y subcategorías mediante un árbol.	Vista jerárquica o cálculo recursivo.
Persistencia	Se permite usar memoria, JSON, CSV, SQLite u otra opción simple.	Datos conservados y justificados.

Componentes gráficos obligatorios en el frontend

- Vista principal con tabla o tarjetas que muestre al menos: título, autor, categoría, año y disponibilidad.
- Formulario para registrar y editar libros.
- Barra de búsqueda visible.
- Control para seleccionar criterio de ordenamiento.
- Componente gráfico para mostrar la jerarquía de categorías en forma de lista anidada o árbol.
- Mensajes claros para operaciones exitosas y errores.

Endpoints sugeridos

- GET /libros
- GET /libros/{id}
- POST /libros
- PUT /libros/{id}
- DELETE /libros/{id}
- GET /libros/buscar?texto=...
- GET /libros/ordenar?campo=...&algoritmo=...
- GET /categorias/arbol

3. Requerimientos algorítmicos

El valor diferencial del proyecto no está solo en que funcione, sino en que el grupo evidencie decisiones correctas sobre búsqueda, ordenamiento, recursión y complejidad.

Estructuras de Datos y Algoritmos 1

Tema	Exigencia mínima	Ejemplos válidos	Qué deben explicar
Ordenamiento	Implementar al menos un algoritmo propio.	Merge Sort, Quick Sort, Insertion Sort, Selection Sort, Bubble Sort.	Por qué lo eligieron y su complejidad temporal.
Búsqueda	Implementar al menos una búsqueda explícita.	Búsqueda lineal o binaria.	Cuándo funciona correctamente y costo temporal.
Recursión	Usar al menos una solución recursiva real.	Recorrer árbol de categorías o contar libros en subcategorías.	Caso base, llamada recursiva y complejidad.
Eficiencia	Justificar si la solución es adecuada para el caso.	Comparación entre alternativas.	Ventajas, limitaciones y mejora posible.

Restricción importante

Los algoritmos evaluados no deben limitarse únicamente al uso de funciones incorporadas del lenguaje. Las funciones nativas pueden usarse como apoyo, pero la lógica de búsqueda, ordenamiento y recursión que se va a calificar debe estar implementada y explicada por el grupo.

Volumen mínimo de datos

- Al menos 25 libros cargados en el sistema.
- Al menos 3 categorías principales y al menos una rama con subcategorías.
- Datos suficientemente variados para que la búsqueda y el ordenamiento tengan sentido.

4. Entregables

Entregable	Contenido mínimo
Repositorio GitHub	Código del frontend y backend, estructura organizada y commits del equipo.
README	Descripción del proyecto, instalación, ejecución, tecnologías, estructura y ejemplos de uso.
Informe breve	Puede estar en PDF o en el README. Debe incluir estructura usada, algoritmos implementados y análisis de complejidad.
Dataset	Archivo o mecanismo de persistencia con los datos de prueba.

Estructuras de Datos y Algoritmos 1

Presentación	Diapositivas breves para explicar problema, arquitectura, demo y análisis.
Sustentación	Demostración funcional del sistema y explicación por parte de todos los integrantes.

Momento	Enfoque	Meta sugerida
Semana 1	Diseño y base del proyecto	Definir estructura, persistencia, endpoints y primera versión del frontend.
Semana 2	Algoritmos y cierre	Terminar búsqueda, ordenamiento, recursión, pruebas, README y presentación.

5. Sustentación y criterios de evaluación

La entrega final deberá ir acompañada de una presentación breve y una demostración funcional del sistema. Se recomienda que todos los integrantes participen activamente en la exposición.

- Duración sugerida de la sustentación: 8 a 10 minutos por grupo.
- Cada integrante debe explicar al menos una parte técnica del proyecto.
- Durante la sustentación el docente puede pedir que se explique una ruta, un componente del frontend, una función recursiva o el análisis de complejidad.

Evaluación alineada con la rúbrica del curso

Criterio	Qué se observará en el proyecto	Para un nivel sobresaliente
Funcionamiento	La aplicación produce las salidas correctas y las muestra en los componentes solicitados del frontend.	Información correcta, completa y visualizada en los componentes gráficos requeridos.
Análisis	El grupo identifica la complejidad del algoritmo de búsqueda, del algoritmo de ordenamiento y de la solución recursiva.	El procedimiento es correcto y tiene máximo un error en la argumentación.
Eficiencia	La lógica general del aplicativo y los procesos de búsqueda y ordenamiento son apropiados para el caso.	La lógica y los procesos de búsqueda/ordenamiento son eficientes y están bien justificados.

Nota pedagógica

Un proyecto final sólido no necesita tener decenas de funcionalidades. Es mejor entregar un sistema pequeño, estable y bien explicado, que una solución grande pero incompleta o sin análisis algorítmico claro.

6. Recomendaciones finales

- Mantener una estructura clara del repositorio: /frontend, /backend y /docs si lo requieren.
- No dejar el análisis de complejidad para el último día; debe construirse junto con el código.
- Probar la aplicación con datos reales de prueba antes de la sustentación.
- Evitar dependencias o funcionalidades que el grupo no pueda explicar durante la presentación.
- Se permite reutilizar parte del trabajo previo de FastAPI, pero el proyecto final debe ampliar de manera evidente la solución e integrar frontend, análisis y algoritmos.

Entrega y sustentación: dos semanas a partir de la publicación de esta guía